

# Simulation Observation Module (SOM)

**OOF™ Origin Open Foundation™**

Independent Methodological Authority

**Architecture Ecosystem:** Structured Reality Standards™

**Architecture Family:** SIMULOS® — Simulation Governance Architecture

**Parent Standard:** Simulation Execution Standard (SES)

**Operational Layer:** Simulation Execution Governance Layer

**Category:** Governance & Enforcement

**Subcategory:** Simulation Governance

**Type:** Parent Standard Module

**Governed Space:** Simulation Observation

**Version:** 1.0

**Status:** Canonical · Open Module

**Effective Date:** 19 August 2026

**Compatibility:** OOF® Methodology OS™ · GOA™ · ORA™ · VALIDOS® · INTEGROS® · ADIT® · AGA™ · AIG® · ASGA™ · RIS™

**AI-Readable:** Yes

**Authority:** OOF®

**Protection:** MIP® — Methodological Intellectual Property

**Canonical Language:** English (UCL™)

## Governed Space

### Simulation Observation

---

#### Canonical Definition

Simulation Observation Module (SOM) defines the governance framework through which materially relevant states, events, transitions, interactions, anomalies, interventions, failures, recoveries, resource conditions, and runtime behaviors occurring during Simulation Execution are identified, observed, captured, contextualized, and preserved so that the execution record reflects not merely that a simulation ran, but what materially occurred while it was running.

SOM establishes the observation layer of the Simulation Execution Standard (SES).

Its purpose is to ensure that materially relevant simulation behavior cannot disappear merely because the simulation technically completed or produced an output.

---

#### Operational Role

SOM serves as the third internal governance space of the Simulation Execution Standard (SES).

---

The preceding modules establish:

SIM — What exact Simulation Instance has been instantiated?

SECM — How is that execution controlled while it runs?

SOM asks:

What materially relevant events, states, transitions, behaviors, anomalies, and interventions occurred during the execution, and under what execution context did they occur?

The progression is:

## **Instantiated Simulation**

↓

## **Controlled Execution**

↓

## **Observation Requirements**

↓

## **Runtime Observation**

↓

## **Event & State Capture**

↓

## **Anomaly / Intervention Capture**

↓

## **Observation Context**

↓

## **Observation Limitation Assessment**

↓

## **Governed Simulation Observation Record**

The observation record then supports:

Simulation Repetition → Execution Documentation → Simulation Output Governance

# Module Operational Space

SOM governs: observation requirements, observation targets, observable states, observable events, runtime behaviors, system transitions, state transitions, scenario transitions, environment transitions, agent behavior, human-system interaction, subsystem interaction, multi-agent interaction, control events, execution interventions, anomalies, unexpected behavior, failure events, degradation events, recovery events, threshold crossings, timing events, synchronization events, resource conditions, computational conditions, communication conditions, sensor conditions, execution-state changes, observation timing, observation frequency, observation resolution, event-triggered observation, threshold-triggered observation, continuous observation, sampled observation, selective observation, observation blind spots, missing observations, observation loss, observation uncertainty, observation provenance, observation traceability, and Governed Simulation Observation Record formation.

## Observation Requirement

An Observation Requirement defines what materially relevant behavior or state must be observable during Simulation Execution.

It answers:

What must we be able to see, detect, capture, or reconstruct during this execution in order to understand what materially occurred?

Observation Requirements may arise from: Simulation Purpose, Scenario Requirements, Edge Cases, Stress Conditions, system boundaries, expected transitions, known failure modes, recovery behavior, autonomous decision states, human interventions, dependencies, timing conditions, resource limits, or previous simulation findings.

## Observation Target

An Observation Target is the specific state, event, object, interaction, process, or condition that the simulation must observe.

Targets may include: Simulation Object state, model state, environment state, subsystem state, agent state, sensor state, communication state, energy state, computational state, control state, autonomy state, authority state, human intervention, failure event, recovery event, or another materially relevant execution condition.

## Observation ≠ Logging Everything

SOM does not require every internal variable to be logged continuously.

Observation depth should reflect the Simulation Purpose and material execution questions.

A simulation may generate enormous amounts of technical telemetry that provide little governance value.

SOM therefore asks:

Which observations are materially necessary to understand execution behavior?

## Observation Materiality

An observation is material where failure to capture it could materially weaken understanding of: why the simulation behaved as it did, whether a scenario condition actually occurred, whether a threshold was crossed, whether a system degraded, whether an intervention changed behavior, whether failure or recovery occurred, whether outputs can be interpreted, or whether execution can later be reconstructed.

## Observation Context

A materially relevant observation should preserve enough context to determine:

### What happened

### When it happened

### Where in the execution it happened

### Under which Scenario

### Under which Simulation Configuration

### Under which input and parameter conditions

and where relevant:

What happened immediately before and after it.

## Observation Identity

A material observation may receive an identifiable reference.

For example:

OBS-00421

An Observation Identity can connect:

Execution ID → Observation ID → Event / State → Output or Downstream Analysis  
Observation Timestamp

Where timing matters, each observation should preserve a relevant temporal reference.

Possible references include: simulation time, wall-clock time, mission time, event time, system-relative time, or another defined temporal basis.

## Event Observation

An Event Observation captures the occurrence of a materially relevant event.

Examples include: collision, threshold crossing, communication loss, sensor failure, mode transition, authority transfer, resource depletion, recovery initiation, emergency response, or scenario termination.

## State Observation

A State Observation captures the condition of a simulated object or system at a defined moment or interval.

Examples include:

Nominal

Degraded

Fallback

Critical

Failed

Recovering

or another simulation-specific state.

## State Transition Observation

SOM gives particular importance to transitions.

A system may behave correctly in two stable states while exhibiting unexpected behavior during the change between them.

Therefore:

## State A → State B

may require explicit observation rather than only observing:

State A

and:

State B.

## Transition Context

A transition record may preserve: originating state, destination state, trigger, transition time, authority involved, relevant inputs, relevant resource state, intermediate behavior, and resulting state.

## Threshold Observation

Simulation behavior may change when a threshold is crossed.

SOM may observe:

## Threshold – $\Delta$



## Threshold Reached



## Threshold + $\Delta$

where behavior around the threshold matters.

## Threshold Crossing Event

A Threshold Crossing Observation may record: threshold identifier, relevant variable, crossing direction, crossing rate, time, resulting state, and downstream behavior.

## Anomaly Observation

An Anomaly Observation captures simulation behavior that differs materially from expected, baseline, or previously characterized behavior.

An anomaly may include: unexpected state, unexpected sequence, unexpected delay, contradictory behavior, output discontinuity, unstable oscillation, resource spike, control conflict, unexpected fallback, or another materially unusual execution condition.

## Anomaly $\neq$ Failure

SIMULOS® establishes:

An anomaly is an observed deviation from expected or characterized behavior; it does not automatically establish system failure.

The consequence requires downstream analysis.

## Unexpected Behavior

Unexpected behavior should remain part of the execution record even where: it was not part of the original Scenario Requirement, it does not immediately affect the final output, or the simulation later recovers.

Unexpected behavior may create: new Scenario Requirements, new Edge Cases, new Stress Conditions, or new Execution Controls.

## Emergent Behavior Observation

In complex simulations, especially: AI, autonomous systems, multi-agent systems, robotics, and adaptive simulations,

behavior may emerge from interactions rather than from one explicitly programmed event.

SOM allows such behavior to become a governed observation without presuming what it means.

## Failure Observation

A Failure Observation records the occurrence of a materially relevant failure condition during simulation.

It may identify: failure type, affected object, time, precursor state, trigger, dependent failures, resulting system state, and recovery behavior.

## Failure ≠ Execution Failure

SOM distinguishes:

### Failure of the simulated system

from:

Failure of the simulation execution itself.

A spacecraft failure inside a correctly executed mission simulation may represent valuable simulation behavior.

A simulation engine crash is an Execution Deviation.

These are different governance objects.

## Degradation Observation

A system may deteriorate before failing.

SOM therefore preserves intermediate states such as:

Nominal

↓

## Performance Reduction

↓

Degraded

↓

Restricted

↓

Critical

↓

Failure

where relevant.

## Recovery Observation

Recovery behavior may be as important as failure.

SOM may observe: recovery trigger, recovery initiation, fallback activation, state restoration, partial restoration, unsuccessful recovery, repeated recovery, and recovery duration.

## No Recovery-by-Output Principle

SIMULOS® establishes:

A final successful state does not prove that the recovery path was acceptable or stable.

Material recovery behavior should remain observable.

## Intervention Observation

SECM governs runtime interventions.

SOM records what materially occurred when those interventions were applied.

An Intervention Observation may identify: intervention type, authority, timing, target, system state before intervention, system state after intervention, and immediate execution effect.

## Human Intervention Observation

Human-in-the-loop simulations may require observation of: action, decision time, response delay, override, missed intervention, incorrect intervention, repeated intervention, and interaction with autonomous behavior.

## Autonomous Decision Observation

Autonomous-system simulations may need to observe: decision state, selected action, escalation, fallback, override request, authority transition, action suppression, or refusal to act

where these are materially relevant and observable.

SOM does not govern the legitimacy of the autonomous decision itself.

It governs the observation of what occurred.

## Multi-Agent Interaction Observation

Multi-agent simulation may require observation of: cooperation, coordination, competition, conflict, communication, resource allocation, deadlock, emergent group behavior, or agent-state propagation.

## Dependency Observation



Where the Simulation Object relies upon dependencies, SOM may observe changes such as:

Available

↓

Degraded

↓

## Intermittent

↓

Unavailable

↓

Recovered

and the system behavior associated with each state.

## Resource Observation

Material resources may include: compute, memory, bandwidth, power, fuel, storage, cooling, time, or another resource.

Resource observations can reveal whether simulation behavior changes as availability changes.

## Computational Observation

For AI and autonomous systems, materially relevant execution observations may include: inference latency, compute saturation, memory pressure, queue growth, scheduling delay, processing backlog, model switching, or dropped execution.

## Communication Observation

Communication observations may include: latency, bandwidth, interruption, packet loss, asymmetric availability, message order, synchronization state, delayed command, and recovery.

## Sensor Observation

Sensor-related observation may capture: confidence, disagreement, degradation, delay, noise, dropout, calibration change, or loss.

## Observation Frequency

Observation Frequency determines how often a target is observed.

Possible approaches include:

Continuous

Periodic

## Event-Triggered

## Threshold-Triggered

## State-Triggered

Sampled

or:

On-Demand.

## Observation Resolution

Observation Resolution determines the level of detail at which execution behavior is preserved.

Too little resolution may hide: rapid transitions, short failures, threshold crossings, oscillations, or brief intervention effects.

Excessive resolution may create enormous data volume without additional material insight.

## Observation Sufficiency

Observation Sufficiency asks:

Is the observation configuration capable of capturing the materially relevant behavior required by the Simulation Purpose and Scenario?

## Observation Blind Spot

An Observation Blind Spot exists where materially relevant execution behavior occurs in a region that cannot be sufficiently observed.

Possible causes include: uninstrumented internal state, inaccessible subsystem, insufficient sampling frequency, missing telemetry, distributed timing mismatch, tool limitation, or simulation architecture limitation.

## Blind Spot $\neq$ No Event

SIMULOS® establishes:

Failure to observe a materially relevant event does not establish that the event did not occur where the observation capability was insufficient.

## Observation Gap

An Observation Gap exists where required observation is: missing, incomplete, delayed, corrupted, inaccessible, or insufficiently resolved.

## Observation Loss

Observation data may be lost because of: logging failure, resource saturation, network failure, storage failure, execution interruption, or technical limitation.

Material loss should remain explicit.

## Observation Uncertainty

Observations may themselves carry uncertainty.

Examples include: timestamp uncertainty, ambiguous state detection, incomplete agent-state visibility, noisy telemetry, approximate classification, or incomplete event reconstruction.

Observation uncertainty should not silently become certainty in downstream analysis.

## Observation Provenance

Observation Provenance establishes where the observation originated.

Possible origins include: simulation engine, model, environment, hardware sensor, software logger, external monitoring system, human observer, AI monitoring agent, or another observation source.

## Observation Source ≠ Interpretation Authority

The fact that a system generated an observation does not establish that its interpretation is correct.

SOM preserves the observation.

Downstream governance determines what it supports.

## Observation Traceability

SOM should support:

### Execution ID

↓

### Scenario

↓

### Simulation State

↓

## Observation Target



## Observed Event / State



Time



Context



## Observation Record



## Output / Downstream Analysis Reference Observation Sequence

Individual observations may have limited meaning in isolation.

SOM may preserve event chains such as:

## Sensor Confidence Decline



## Decision Delay



## Fallback Activation



## Communication Loss



## Autonomous Recovery



## Resource Depletion

This creates a traceable behavior sequence.

## Observation Causality Boundary

SOM does not automatically claim:

A caused B.

It records:

A occurred before B under these execution conditions.

Causal interpretation requires additional analysis.

## No Observation-to-Causality Principle

SIMULOS® establishes:

Temporal sequence and observed association do not automatically establish causal explanation.

## Observation Correlation

Two observations may be correlated.

SOM can preserve the relationship without declaring causality.

## Observation Completeness

Observation Completeness asks whether all materially required Observation Targets were sufficiently captured during execution.

Possible statuses may include:

Complete

### Conditionally Complete

### Partially Complete

Incomplete

or:

## Undetermined. Observation Completeness ≠ Execution Completeness

A simulation may execute completely while its observation record remains incomplete.

Conversely, an interrupted execution may still have complete observations for the period actually executed.

## Observation Status

A material observation may receive a status such as:

Captured

## Partially Captured

Delayed

Uncertain

Lost

## Not Observable

or:

Observation Status Undetermined.

## Observation Reassessment

Observation strategy may require change where: new anomalies are discovered, scenario complexity increases, system architecture changes, new Edge Cases emerge, new Stress Conditions are introduced, or previous runs reveal blind spots.

## Adaptive Observation

A simulation may dynamically increase observation around: anomalies, thresholds, failures, transitions, or high-sensitivity regions.

For example:

## Normal Sampling

↓

## Threshold Approached

↓

## High-Resolution Observation Activated

This can improve informational value without requiring maximum telemetry continuously.

## No Hidden Adaptive Observation Principle

Where observation behavior changes dynamically, the transition in observation mode should remain traceable.

## Observation and Repetition

SOM provides the runtime observation basis consumed by:

SRPM — Simulation Repetition Module.

Repeated simulations may reveal: consistent behavior, variable behavior, stochastic behavior, rare anomalies, inconsistent transitions, or execution-sensitive outcomes.

Without controlled observation, repetition may generate different outputs without revealing why.

## Observation and Execution Documentation

SOM observations become part of the record governed by:

SEDM — Simulation Execution Documentation Module.

SEDM determines how observations are preserved within the complete execution record.

## Observation and Simulation Output Governance

SOGS governs Simulation Outputs.

SOM governs what occurred during execution.

These may differ.

For example:

Simulation Output: Mission completed.

Observation: Autonomous fallback occurred three times, power margin reached a critical threshold, and one recovery attempt failed before final recovery.

The final output alone may conceal materially important execution behavior.

This distinction is one of SOM's core purposes.

## No Final-Output Sufficiency Principle

SIMULOS® establishes:

A final Simulation Output must not be assumed to represent the complete material behavior of the execution that produced it.

## Space and Mission Systems

Space simulations frequently contain short, critical events embedded within long mission runs.

Examples include: stage separation, docking, navigation transition, communication blackout, safe-mode entry, autonomous recovery, power reconfiguration, landing sequence, or subsystem failover.

A mission may finish successfully while experiencing transient behavior that materially affects mission robustness.

SOM provides a structure for preserving those events rather than reducing the execution to:

Mission Success / Mission Failure. Use Case 1 — Spacecraft Autonomous Recovery

## Scenario

A spacecraft simulation enters autonomous recovery after navigation confidence deteriorates.

The final mission output is:

## Recovery Successful. Application

SOM observes: navigation-confidence decline, communication latency, autonomous recovery initiation, subsystem shutdown, power draw, fallback transition, temporary loss of attitude-control margin, recovery stabilization, and final mission state.

## Result

The organization sees that recovery succeeded, but also that the execution passed close to a power and control boundary.

The observation record creates a new candidate Stress Condition for later simulation.

Without SOM, the final output:

## Recovery Successful

could hide the material execution bottleneck. Use Case 2 — Autonomous Vehicle Perception Conflict

## Scenario

An autonomous vehicle completes a pedestrian-avoidance simulation without collision.

The final output is:

## Collision Avoided. Application

SOM captures: camera-confidence decline, radar disagreement, perception reclassification, braking delay, trajectory correction, fallback-state activation, and clearance distance.

## Result

The execution is technically successful, but observation reveals that collision avoidance depended upon a late fallback transition.

The organization can now distinguish:



successful final outcome

from:

robust execution behavior. Use Case 3 — AI Agent Multi-Tool Execution

## Scenario

An AI agent completes a complex workflow using multiple external tools.  
Application

SOM observes: tool invocation sequence, tool latency, permission state, conflicting tool output, memory-state change, action suppression, retry behavior, escalation, and final completion.

## Result

The simulation reveals that the agent reached the correct final result only after an internal authority conflict and repeated tool calls.

That behavior becomes visible for downstream scenario and execution governance.

## Existing Simulation Program Diagnostic

SOM can be applied to an existing simulation program.

An organization may ask:

What exactly do we observe during execution?

Which materially relevant states are invisible?

Do we observe transitions or only final states?

Do we preserve anomalies?

Can we distinguish simulated-system failure from simulation-engine failure?

Can we reconstruct the sequence leading to an output?

Do we preserve failed recovery attempts?

Are our observation intervals too coarse to detect short events?

Which simulation behaviors disappear because our logging system never captures them?

This can reveal that the simulation program's bottleneck is not execution capacity but observation capability.

## Module Boundary

SOM begins when an instantiated and controlled Simulation Execution produces materially observable states, events, interactions, transitions, and behaviors.

SOM ends when materially relevant execution behavior has been sufficiently: observed, identified, contextualized, temporally referenced, linked to execution state, bounded by observation limitations, and preserved

as a Governed Simulation Observation Record.

SOM does not govern: Scenario Identification, Simulation Instantiation, general Execution Control, final Simulation Output classification, Simulation Inference, or Validation Determination.

---

## Minimum Implementation Framework

### 1. Identify Observation Requirements

Determine which execution states, events, transitions, interactions, resources, failures, and recoveries must remain observable. 2. Define Observation Targets

Identify what must be monitored during execution. 3. Establish Observation Method

Define whether observation is: continuous, periodic, event-triggered, threshold-triggered, state-triggered, or another justified form.

### 4. Establish Observation Context

Bind observations to: Execution ID, Scenario, time, state, model/environment configuration, and applicable input conditions.

### 5. Capture Material Runtime Behavior

Preserve material: events, transitions, anomalies, failures, recoveries, interventions, interactions, and resource states.

### 6. Identify Observation Limitations

Record: blind spots, missing telemetry, resolution limits, observation loss, uncertainty, and inaccessible states.

### 7. Assess Observation Completeness

Determine whether the materially required Observation Targets were sufficiently captured. 8. Preserve Observation Traceability

Maintain:

**Execution → State → Observation → Context**  
**→ Sequence → Downstream Reference**

---

## Governance Outputs

SOM may produce: Observation Requirement, Observation Target Register, Simulation Observation Identifier, Simulation Observation Record, State

Observation Record, Event Observation Record, State Transition Record, Threshold Crossing Record, Runtime Behavior Record, Anomaly Observation Record, Unexpected Behavior Record, Emergent Behavior Record, Failure Observation Record, Degradation Observation Record, Recovery Observation Record, Intervention Observation Record, Human Intervention Observation Record, Autonomous Decision Observation Record, Multi-Agent Interaction Observation Record, Dependency Observation Record, Resource Observation Record, Computational Observation Record, Communication Observation Record, Sensor Observation Record, Observation Frequency Record, Observation Resolution Record, Observation Blind-Spot Record, Observation Gap Record, Observation Loss Record, Observation Uncertainty Record, Observation Provenance Record, Observation Sequence Record, Observation Completeness Assessment, Observation Status, Adaptive Observation Record, Simulation Observation Traceability Map, and Governed Simulation Observation Record.

## Architectural Position

Simulation Observation Module (SOM) is the third internal governance space of the Simulation Execution Standard (SES).

The complete SES progression is:

## Simulation Instantiation

↓

## Simulation Execution Control

↓

## Simulation Observation

↓

## Simulation Repetition

↓

## Simulation Execution Documentation

SIM establishes:

What exact execution instance exists?

SECM establishes:

How is it controlled while running?

SOM establishes:

What materially occurred during that execution?

SRPM then establishes:

What happens when execution is repeated, and under what comparability

conditions?

SEDM finally establishes:

Can the complete execution history be reconstructed?

## Foundational Principle

A simulation execution is not sufficiently understood merely because it reaches a final output; materially relevant states, transitions, anomalies, interventions, failures, recoveries, and runtime conditions must remain observable enough to reconstruct what actually occurred during the path to that output.

---

## Canonical Closing Statement

Simulation Observation Module establishes the runtime visibility layer of the Simulation Execution Standard by governing how materially relevant execution states, events, transitions, anomalies, interventions, failures, recoveries, resource conditions, system interactions, autonomous behaviors, and runtime sequences are identified, captured, contextualized, and preserved. Through Observation Requirements, target definition, timing, resolution, state and event capture, blind-spot identification, observation uncertainty, provenance, completeness assessment, and full execution traceability, SOM prevents simulation programs from reducing complex execution behavior to final outputs alone and ensures that the path through which a simulation produced its result remains visible enough to support repetition, documentation, downstream output governance, validation, and simulation improvement.

---

# OOF® MIP® Protection Notice

## OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

**Active Protection & Compatibility Detection applies.**

## Canonical Source

<https://originopenfoundation.org/>